

PA1 Report: Beyond i.i.d.

Syed Ramiz Abbas

LUMS

Advanced Techniques in ML (EE-5102 / CS-6304) - October 1, 2026

Abstract

This report reflects our study of four different ways a model's training assumptions can fail to hold at test time. Task 1 asks which visual cues (shape, texture, color, or position) different pretrained backbones actually rely on, and finds that ResNet-50 relies on texture about as much as on shape while ViT and CLIP are strongly shape-biased, yet react very differently to patch-order corruption despite sharing the ViT-B architecture. Task 2 studies unsupervised domain adaptation on PACS and finds that CDAN can push its domain classifier down to chance while doing far worse on the target domain (and that DANN's adversarial training becomes unstable), showing that better domain alignment does not necessarily lead to better target accuracy. Task 3 studies domain generalization to a domain never seen during training, and finds the same decoupling for a sharpness-based regularizer (SAM): the flattest loss landscape does not produce the best held-out accuracy. Task 4 studies open-set recognition on CIFAR-10/100, and finds that ordinary closed-set classifiers need deliberately chosen machinery, PROSER's placeholder score or Mahalanobis distance in particular, to do noticeably better at rejecting unseen classes, and that detection is far harder for semantically close unknowns than distant ones. Across tasks 2 and 3, we found that improving a secondary objective, whether domain confusion or loss flatness, does not necessarily lead to better performance on the actual task. Code: <https://github.com/sramiza/pa1-beyond-iid>

Introduction

Most machine learning courses lead us to assume that our training data and our test data come from the same distribution; meaning, the same kind of images and the same set of possible labels; no surprises. In practice this assumption does not hold, and it can break in several different ways. This report is about four different ways it can break, each explored in its own task, ordered from the milder one to the more severe break. Task 1 keeps the label set and the underlying distribution the same, but asks a more subtle question: which visual cue within a single dataset is a model actually using to make its decision: shape, texture, color, or where an object sits in the frame, and how fragile is that choice? Task 2 moves to unsupervised domain adaptation: we still want to predict the same set of labels, but now the images we test on look different from the images we trained on (e.g. sketches instead of photos), and we don't get any labels for the new-looking images; instead, only the images themselves. The question is how much of that gap can be closed using clever training tricks with no target labels at all. Task 3 makes this harder: we don't get to see even unlabeled examples from the new domain during training. We train on a few source domains and then test whether the model can generalize to a domain it has never seen. We also examine whether explicitly encouraging a flatter loss landscape helps compared with standard training.

Task 4 changes the problem entirely: instead of the label set staying fixed, the model now has to deal with test images that belong to classes it has never seen at all. The task stops being "which of these classes is this" and becomes "is this even one of my classes in the first place:" recognition versus rejection. The Discussion section ties these back together, since although they were run as four separate experiments, several of the same underlying lessons show up more than once.

Task 1: Inductive Biases and Feature Representations

Methods

We take three neural networks that already come pretrained and never update their weights any further ("frozen"). A ResNet-50 (a standard convolutional network, trained on ImageNet), a ViT-B/16 (a Vision Transformer, also trained on ImageNet), and CLIP's ViT-B/32 (a Vision Transformer trained differently; not to classify ImageNet, but to match images with captions). We test them on **STL-10**, a 10-class image dataset, because its classes are visually distinct and roughly balanced, which lets us see architecture-driven biases without also fighting a hard fine-grained-classification problem. Each network turns an image into a single feature vector in its own way. ResNet-50 pools its final convolutional feature map, ViT-B/16 uses its final "class token," and CLIP uses its normalized image embedding, but all three see exactly the same raw, unnormalized 224x224 pixel input; each model's own required normalization is only applied right before that model's own forward pass, so every model reacts to pixel-identical images rather than images preprocessed differently per model.

For each backbone, we keep its weights frozen and train a single linear layer on top of its extracted features (a stratified 80/20 split of the official training partition, seed 6304; AdamW, learning rate $1e-3$, weight decay $1e-4$, up to 50 epochs, stopping early if validation accuracy hasn't improved in 5 epochs). CLIP is also evaluated "zero-shot", with no trained head at all, just by comparing the image embedding against text embeddings of the prompt "a photo of a {class}." for every class name. All evaluation uses a fixed, class-balanced set of 500 test images (50 per class, seed 6304); STL-10's test set already has 800 images per class, so this subsample doesn't introduce any imbalance.

Color bias: we test two color interventions. Converting images to grayscale, and a fixed hue rotation (shifting every color by the same fixed amount, `hue_factor=0.3`). We picked hue rotation over swapping in an entirely different color palette because it isolates exactly one thing, the hue channel, while leaving brightness, which carries shape and texture, untouched.

Hypothesis: hue rotation should hurt more than grayscale for any class whose color is actually diagnostic, since grayscale keeps the relative brightness contrast a shape-based decision could still use, while hue rotation actively feeds the model a wrong color.

Metric: change in accuracy, and how often the transformed prediction matches the clean prediction.

Shape vs. texture (cue-conflict): to directly pit shape against texture, we generate cue-conflict images the way Geirhos et al. (2019) did: take a photo of one class (say, a cat) and repaint its texture so it now looks like it's made of a different class's texture (say, truck). If a model predicts "cat" it's using shape; if it predicts "truck" it's using texture. We use Gatys et al.'s original optimization-based style transfer: running gradient descent directly on the output image's pixels, with a pretrained VGG-19 defining a content loss (keep the original shape) and a style loss (match the new texture's statistics), rather than a faster AdaIN-style network, since AdaIN needs an extra pretrained decoder we couldn't reliably download inside a Kaggle session. Gatys' method only needs the standard torchvision VGG-19, and gives us direct, tunable control over **style strength** (`style_weight=1e6` vs. `content_weight=1`, set high enough that the texture

actually matters). We use five class pairs (cat–truck, dog–airplane, horse–ship, bird–car, deer–monkey), stylized in both directions, 30 images generated per direction. Before any model sees these images, we run an automatic check (SSIM between the stylized and original image, plus a near-blank-output check) to throw out any stylization that clearly failed (all 300 passed, so none were thrown out).

Hypothesis (Geirhos et al., 2019): the convolutional network (ResNet) should show a lower shape bias than the two transformer-based models.

Metric: **Shape Bias%** (of the decisive predictions, how many agreed with shape) and **Coverage%** (what fraction of predictions were decisive at all, i.e. matched either shape or texture).

Translation: we shift each image by 0/8/16/32 pixels in each of the four directions (padding the edges by reflection so nothing goes blank), and average the result across directions. **Patch structure**: we cut each image into a 4x4 grid and shuffle the pieces with one fixed, non-identity shuffle per image (seed 6304), using the exact same shuffled image for all three backbones so the comparison is fair.

Representation analysis: for each of the four required interventions, we compute a stability score I_T : the cosine similarity between a model's feature vector for the clean image and its feature vector for the transformed image (1.0 means the representation didn't move at all). We also visualize the features with t-SNE (sklearn.manifold.TSNE, perplexity 30, 1000 iterations, random_state=6304), a dimensionality-reduction method that tries to keep points that were close together in the original high-dimensional space close together in a 2D plot, chosen over UMAP because it specifically emphasizes whether a point stays near its original neighbors, which is exactly what we want to check: does a transformed image still land near its own class's clean examples? Every (backbone, intervention) pair uses the same settings and seed, but a t-SNE plot's absolute coordinates are arbitrary and not comparable across separately-fit plots; only the relative clustering within one plot means anything.

Results

Clean baseline: before testing any interventions, we check that all four setups (three backbones plus CLIP's zero-shot mode) work well on unmodified images. ResNet-50 gets 95.20% accuracy (95.20% F1), ViT-B/16 gets 96.40% (96.41%), CLIP's trained head gets 95.40% (95.37%), and CLIP zero-shot gets 93.40% (93.28%). All four are strong and comparable, so any differences below aren't just an artifact of one model being weak to begin with.

Color bias: removing color entirely (grayscale) costs ResNet and ViT 2.20 accuracy points and costs CLIP (both its trained head and its zero-shot mode) 3.60 points; the hue rotation costs ResNet 2.80, ViT 3.40, CLIP head 3.20, CLIP zero-shot 2.60. So CLIP leans on color the most when color is simply removed, while ResNet and ViT are hurt slightly more by a *wrong* color (hue rotation) than by *no* color at all (grayscale). Those gaps are only 3 and 6 images out of 500, though, so our hypothesis gets at most weak support from ResNet and ViT, and none from CLIP.

Shape vs. texture: ResNet-50 lands almost exactly at Shape Bias = 53.2% (Coverage 36.3%), essentially a coin flip between shape and texture; less extreme than the roughly 22% Geirhos et al. (2019) reported for ResNet-50, but still far more texture-reliant than the transformers. ViT-B/16 comes in at 82.6% (Coverage 57.3%) and CLIP at 77.8%/82.2% (head/zero-shot, Coverage 48.0%/52.3%); both transformer-based models are strongly shape-biased, confirming the hypothesis, and their higher coverage means this number can be trusted more (ResNet's 53.2% rests on fewer decisive predictions, since many of its outputs didn't clearly match either shape or texture).

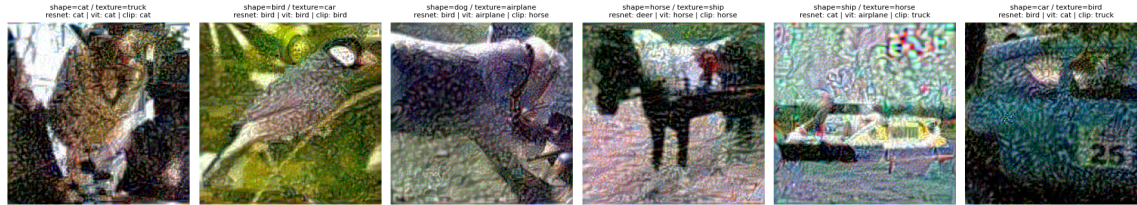


Figure: cue-conflict stimuli (Gatys-style stylization) with each model's prediction. E.g. *shape=dog/texture=airplane*: ResNet predicts "bird" (texture-adjacent confusion), ViT predicts "airplane" (texture), CLIP predicts "horse" (neither cue exactly, but shape-family); *shape=horse/texture=ship*: both ViT and CLIP predict "horse" (shape) while ResNet predicts "deer" (shape-family miss, not texture-driven), illustrating that the shape/texture split is real but not absolute for any one backbone.

Translation: all four setups hold up reasonably well under shifting (averaged clean $\rightarrow$ 32px shift: ResNet 95.2% $\rightarrow$ 93.9%, ViT 96.4% $\rightarrow$ 96.1%, CLIP head 95.4% $\rightarrow$ 93.3%, CLIP zero-shot 93.4% $\rightarrow$ 92.3%). ViT is actually the *most* translation-robust of the three, a bit surprising since one might expect a model that explicitly encodes patch position to be more sensitive to position changes than a convolutional network.

Patch shuffle hurts far more than translation does, but it doesn't change which model looks most robust; it mostly widens the gaps. ResNet drops 10.80 points (86.6% of predictions stay the same), ViT drops only 7.60 points (90.6% consistency, the *most* robust of the three), CLIP's head drops 22.80 points (73.8% consistency, the *least* robust), and CLIP zero-shot drops 15.80 points. ViT being the *most* robust to scrambled patches while CLIP, also a ViT-B (but with 32-pixel patches instead of 16), is by far the *least* robust tells us this isn't simply a "transformers beat CNNs" story. Something beyond the architecture family matters here, but we can't say which part: ViT and CLIP also differ in patch size (16 vs. 32), pretraining data and pretraining objective. One possible reading is that classifying ImageNet leaves ViT's features more tolerant of patches being out of order than CLIP's contrastive image-caption training does, but our setup can't separate that from the other differences.

Representation stability (I_T): under grayscale: ResNet 0.798, ViT 0.732, CLIP 0.885; under 16px translation (a single upward shift, not an average over directions): all three stay above 0.95; under patch shuffle: ResNet 0.694, ViT 0.627, CLIP 0.750. The patch-shuffle numbers: **ViT's internal representation moves the most (lowest $I_T=0.627$), yet its predictions are the most stable of the three; CLIP's representation moves the least (highest $I_T=0.750$), yet its predictions are the least stable.** How much a model's internal feature vector shifts and how much its final answer changes turn out to be two separate questions here. A big shift in feature space doesn't necessarily cross into a different predicted class (ViT's case), while a comparatively small shift can (CLIP's case); most plausibly because CLIP's embedding space packs classes more tightly together, so even a modest shift is more likely to land in a neighboring class's territory.

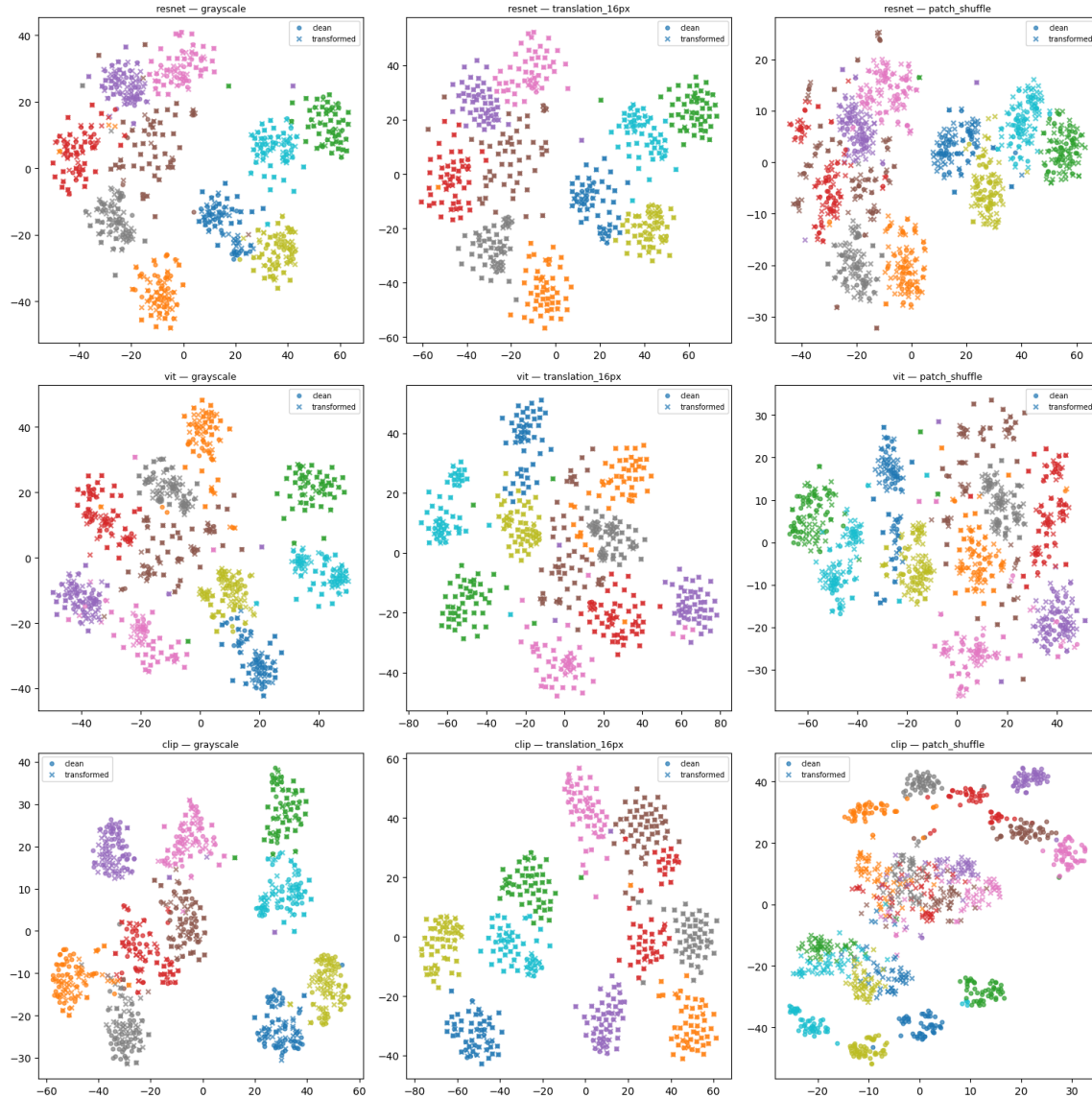


Figure: *t*-SNE projections of clean (circle) vs. transformed (x) features, colored by class. Grayscale and translation leave every backbone's clusters essentially intact (clean and transformed points overlap within each color); patch shuffle visibly loosens ResNet's clusters and fragments CLIP's the most, which matches the prediction drops above but not the I_T numbers (CLIP has the highest patch-shuffle I_T , 0.750), the same mismatch discussed above.

Cue-conflict I_T (measured on the first 50 cue-conflict images, which only cover the cat and truck content classes, each compared against one fixed same-class photo rather than its own literal pre-stylization source, a documented approximation): ResNet 0.126, ViT 0.053, CLIP 0.588. All three shift far more here than under any other intervention. Cue-conflict is the only intervention that actively injects evidence for a *second, competing* class, rather than simply degrading or scrambling the first.

Research-question answers

1. Putting the color and cue-conflict results together: CLIP is both the model hurt most when color is removed (Color bias, above) *and* one of the two most shape-biased models (Shape vs. texture,

above); so for CLIP, leaning on color and leaning on shape aren't mutually exclusive. ResNet's low cue-conflict coverage (36.3%) means we should be cautious reading too much into its near-50/50 shape/texture split; the transformer-based models' higher coverage makes their strong shape bias the more trustworthy result.

2. Translation is only mildly disruptive, and ViT handles it best. Patch shuffling is far more disruptive, but the ranking mostly stays the same while the gaps get much wider. ViT is still the most robust, and CLIP, already a bit behind ResNet under translation, falls far behind despite sharing ViT's ViT-B architecture.
3. A model's internal representation and its actual predictions disagreeing is demonstrated by the patch-shuffle result above: ViT's features move a lot but its predictions barely change, while CLIP's features barely move but its predictions change a lot. Separately, CLIP's zero-shot predictions track its own trained linear head fairly closely under the color interventions (both drop 3.60 points under grayscale) and on cue-conflict (77.8% vs. 82.2% shape bias), suggesting the trained head didn't learn much beyond what the zero-shot prompt already captured there; the exception is patch shuffle, where the head drops 7 points more than zero-shot (22.80 vs. 15.80).
4. The patch-shuffle result is our clearest case of a difference the architecture family alone can't explain; ViT and CLIP are both ViT-B models yet behave oppositely. We can't pin it on the pretraining objective specifically, though, since they also differ in patch size (16 vs. 32) and pretraining data. The shape-bias result looks more like an architecture-level effect, since both transformer-based models behave similarly to each other and differently from the CNN, though we can't fully rule out that differences in pretraining data or scale between ImageNet-supervised ViT and web-scale CLIP also play a role.

Task 2: Unsupervised Domain Adaptation

Methods

We use the PACS dataset, treating Sketch as the unlabeled target domain we want to adapt to, and Photo, Art Painting, and Cartoon as the three labeled source domains we train on. Following the assignment's protocol, each source domain is split 80/20 (seed 6304), the backbone is a ResNet-18 pretrained on ImageNet with its BatchNorm running statistics kept frozen (so the model doesn't quietly adapt its internal statistics to whichever domain's images happen to be passing through it), and we train with AdamW (learning rate $1e-4$, weight decay $1e-4$), picking the checkpoint with the best mean source-validation macro-F1 and stopping early if that hasn't improved in 5 epochs.

We compare four approaches: **Source-only ERM** (just train normally on the labeled source domains and hope it transfers); **DAN** (add a penalty called MMD that measures how different the source and target feature distributions look overall, and try to shrink that difference, with weight $\lambda=1$, using a mix of Gaussian kernels whose bandwidths are set relative to the median distance between features in each batch); **DANN** (instead of a fixed penalty, train a small "domain classifier" adversarially, through a gradient reversal layer, to try to tell source and target features apart, while the main network tries to fool it); and **CDAN**, a variant of DANN where the domain classifier doesn't just see the raw features, but the features combined with the model's own class predictions ($f \otimes p$), making the alignment class-aware rather than blind to it.

DANN and CDAN turned out to be unstable to train out of the box as the gradient-reversal layer's flipped-sign gradient interacts badly with Adam's per-parameter adaptive scaling, and gradients would blow up. We tried to fix this by loosening AdamW's eps parameter ($1e-4$ instead of the default $1e-8$) and

clipping the gradient norm to a maximum of 1.0. This kept CDAN stable, but DANN still blew up partway through its first epoch: by batch 140 (α about 0.14) its gradient norm before clipping had gone from about 15 to over 1,300 and its domain loss from about 1 to 63, and by epoch 5 gradient norms were in the hundreds of thousands and mean source F1 was down to 5.07%. Early stopping kept DANN's epoch-1 checkpoint, which was already affected (63.44% mean source F1, against 91.02% for ERM after one epoch).

Results

Method	Sketch Acc	Sketch F1	Domain Separability
ERM	72.46%	74.41%	99.73%
DAN	73.58%	71.76%	92.58%
DANN	30.54%	14.34%	100.00%
CDAN	35.71%	37.05%	98.35%

Source-only ERM (no adaptation at all) already reaches 72.46% Sketch accuracy (74.41% macro-F1). DAN basically matches this (73.58%/71.76%), at the cost of a small amount of source accuracy, consistent with the idea that only weakly aligning the overall feature distributions has a limited effect either way. DANN and CDAN both fail badly (30.54%/14.34% and 35.71%/37.05%), and the gap between DANN's accuracy and its macro-F1 shows it collapsing onto essentially one class: it only gets dogs (93.0%) and guitars (79.3%) right, scores 0% on the other five classes, and labels most elephants, giraffes and houses "dog". CDAN doesn't have that gap (its macro-F1 is slightly above its accuracy) because its collapse is only partial: it dumps most elephants (656 of 740) and dogs (662 of 772) into "person", while still getting house, person, guitar and giraffe mostly or partly right. For DANN, that single class's high recall inflates the raw accuracy number, while macro-F1, which averages performance evenly across all seven classes, correctly exposes that the model has essentially stopped trying on five of the seven classes.

We also ran a domain-separability diagnostic: train a simple logistic-regression classifier ($C=1$, on the frozen features, 70/30 split, seed 6304, chance level = 50%) to guess whether a given feature vector came from a source or target image. If a method has truly made the domains "look the same" to the network, this classifier should struggle (near 50%). Instead, separability stayed high across all four methods (92–100%) and didn't track how well the method actually did on the target domain. DANN, the worst-performing method, had the *highest* separability (100.00%), and CDAN, the second worst, was barely below ERM (98.35% vs. 99.73%), while DAN, the only method that meaningfully lowered separability (92.58%), was also the only one to roughly match ERM's target accuracy. Making the domains harder to tell apart did not correspond to better recognition on the target domain here. This lines up with Zhao et al. (2019)'s critique that a domain classifier being "fooled" during adversarial training doesn't guarantee the features are actually invariant in a way an independent probe (like our logistic regression) would agree with.

To see how sensitive DAN is to its alignment strength, we swept its λ_{MMD} weight (0.1, 1, 10). The relationship is not a simple "more alignment is better":

λ_{MMD}	Source F1	Sketch Acc	Sketch F1	Separability
0.1	95.09%	68.52%	63.59%	95.74%

λ _MMD	Source F1	Sketch Acc	Sketch F1	Separability
1	91.86%	73.58%	71.76%	92.58%
10	89.11%	69.25%	65.94%	94.09%

$\lambda=1$ gives the best target accuracy and the lowest separability of the three settings, at some cost in source F1 compared to $\lambda=0.1$. Increasing it to $\lambda=10$ doesn't reduce separability any further (94.09% vs. 92.58% at $\lambda=1$) despite applying 10x the alignment pressure; it just costs more source accuracy for no benefit.

Task 3: Domain Generalization

Methods

We reuse the exact same Source-only ERM checkpoint and source-domain data pipeline from Task 2, unchanged. Importantly, no Sketch data, not even unlabeled Sketch images, is loaded at any point during training, checkpoint selection, or hyperparameter tuning in this task. Sketch is only loaded once, at the very end, for the final evaluation, after every other decision has already been locked in. This matters because Task 3 is specifically about generalizing to a domain the model has genuinely never seen in any form during development.

We compare plain ERM against two methods aimed at generalizing (not adapting) to an unseen domain. **DAN-DG**, which applies the same MMD-style alignment penalty as Task 2's DAN, but now pairwise across the three *source* domains only ($\lambda_{DG}=1$), trying to make the model's features look the same across Photo/Art/Cartoon in the hope that this generalizes to Sketch too; and **SAM** (Sharpness-Aware Minimization, $\rho=0.05$), which doesn't touch the data at all; instead it changes how the optimizer works, explicitly searching for a minimum of the loss function that is "flat" (the loss doesn't spike if you nudge the weights slightly), on the theory that flat minima generalize better than sharp ones. Both methods use the same frozen-BatchNorm policy as Task 2. In the table below, Src Separability is a three-way version of Task 2's probe (which source domain a feature came from), so chance is 33.3% here instead of 50%, and Sharpness (Δ) is how much the cross-entropy on a fixed batch of 96 source-validation images goes up after nudging the weights by a radius of 0.05 in the gradient direction; lower means locally flatter. One caveat: the $\lambda_{DG}=1$ DAN-DG run was cut off during epoch 13, two epochs after its best epoch (11), so the 5-epoch early-stopping rule never fired for it; we use its epoch-11 checkpoint.

Results

Method	Mean Src F1	Worst Src F1	Sketch Acc	Sketch F1	Src Separability	Sharpness (Δ)
ERM	93.99%	91.72%	72.46%	74.41%	89.37%	0.4146
DAN-DG	94.56%	91.94%	71.01%	72.11%	72.43%	0.4690
SAM	94.62%	91.82%	59.38%	65.02%	85.71%	0.2059

Both DAN-DG and SAM do better than plain ERM on their source-domain validation metrics, including worst-domain F1, which is meant to be a conservative, generalization-friendly metric to select on. And yet both do worse than ERM on the actually new Sketch domain. Metrics computed on the source domains, can overestimate how well a model will generalize to a domain it has truly never seen; and if you picked a method using only those source metrics, you'd have picked the worse generalizer both times.

SAM achieved the flattest local minimum by a wide margin ($\Delta_{\text{sharp}}=0.2059$ vs. ERM's 0.4146 and DAN-DG's 0.4690), but produced the worst Sketch performance of the three methods. Per-class analysis shows SAM over-predicting "dog": it is the most common wrong answer for 5 of the 6 other classes, although SAM still gets 39–86% of every class right, so this is a bias rather than a collapse. DANN also ended up predicting "dog" in Task 2, but since that run diverged, we don't count it as independent evidence that "dog" is a special weak point of Sketch.

We ran the same kind of sweep for DAN-DG's λ_{DG} (0.1, 1, 10), and once again $\lambda=1$ comes out on top on every metric except worst-source F1, where $\lambda=0.1$ is slightly higher (92.40% vs. 91.94%):

λ_{DG}	Mean Src F1	Worst Src F1	Separability	Sketch Acc	Sketch F1
0.1	94.02%	92.40%	81.06%	64.83%	66.07%
1	94.56%	91.94%	72.43%	71.01%	72.11%
10	65.03%	56.14%	75.08%	38.05%	37.78%

At $\lambda=10$, training collapses outright: mean source F1 reaches 65.03% after epoch 1 (the checkpoint that gets kept) and then drops to 8.73% by epoch 2. We traced this to a loss-scale imbalance: at $\lambda=10$, the MMD alignment term (roughly 75–90 per epoch before weighting, so roughly 750–900 once multiplied by $\lambda=10$) is already around 9 times larger than the classification loss in the first epoch (about 92–95 per epoch in the $\lambda=0.1$ and $\lambda=1$ runs), and 40–60 times larger than the classification loss of a healthy later epoch (roughly 15–20), so the optimizer essentially stops trying to classify at all and only chases the alignment penalty. Interestingly, $\lambda=10$'s separability score (75.08%) is actually *lower* than $\lambda=0.1$'s (81.06%), even though $\lambda=10$ produces a much weaker classifier (65.03% source F1, 38.05% Sketch accuracy).

Task 4: Open-Set Recognition (CIFAR-10/100)

Methods

We study open-set recognition, the problem of a classifier correctly saying "I don't recognize this" for inputs from classes it's never seen, using CIFAR-10 as our 10 known classes, and two groups of unknown classes drawn from CIFAR-100. A *near* group (10 classes semantically similar to something in CIFAR-10: bear, leopard, lion, tiger, wolf, pickup_truck, streetcar, tractor, tank, rocket) and a *far* group (10 classes semantically distant from anything in CIFAR-10: orchid, poppy, rose, sunflower, tulip, clock, keyboard, telephone, wardrobe, table). Each group is subsampled to exactly 800 images. CIFAR-10 itself is split 90/10 (stratified, seed 6304) into training and validation, with the official CIFAR-10 test set held out separately for both closed-set accuracy and open-set (AUROC) evaluation.

All three classifiers below share the same backbone. A CIFAR-ResNet-18 (a ResNet-18 adapted for small 32x32 images: a 3x3, stride-1 first layer and no initial max-pooling, so we don't throw away resolution we can't spare), and the same base training recipe. SGD (learning rate 0.1, momentum 0.9, weight decay $5e-4$), a cosine-annealed learning rate schedule, batch size 128, seed 6304, saving a checkpoint every time validation accuracy hits a new best, with no early stopping.

- **Vanilla:** just the base recipe above, trained for 100 epochs with standard random-crop-and-flip data augmentation.
- **GCSC:** identical recipe and length, but with RandAugment (a stronger, randomized data augmentation, num_ops=2, magnitude=9) in place of standard augmentation. The idea, from

Vaze et al. (2022), is that open-set performance tracks closed-set accuracy, so a stronger closed-set training recipe should also make the model better at telling unknowns apart from knowns.

- **PROSER**: starts from the Vanilla checkpoint and adds 5 randomly-initialized extra "dummy" output units to the classifier, then fine-tunes for 50 epochs (at a lower learning rate of 0.01, cosine-annealed over those 50 epochs) with two extra loss terms meant to teach those dummy units to represent "none of the above". A classifier-placeholder loss ($\beta=1$) that pushes the dummy units to act like a plausible runner-up class for every real training example, and a data-placeholder loss ($\gamma=0.1$) that uses manifold mixup. Blending the internal features of two different-class examples together (mixing weight $\lambda \sim \text{Beta}(2,2)$, blended at the boundary between layer2 and layer3), and pushes that blended, in-between example toward activating the dummy units instead of a real class. The goal is to synthesize a rough stand-in for "unknown" examples without ever training on real unknown data.

We evaluate four **post-hoc** novelty scores, meaning none of them require any extra training, they're just different ways of turning the Vanilla model's existing logits or features into a single "how in-distribution does this look" number: **Maximum Softmax Probability (MSP)** (the model's own top confidence), **Maximum Logit Score (MLS)** (the top raw logit, before the softmax squashes it), **Energy** (the log-sum-exp of all the logits, a score borrowed from energy-based models), and **Mahalanobis distance** (how far a feature vector sits from the nearest class's mean, accounting for how spread out the features are, using a shared diagonal covariance (one for all classes) estimated from unaugmented training features, with a small $+1e-6$ jitter for numerical stability). GCSC is scored with MLS; PROSER is scored with both MLS (on its normal, known-class logits) and its own dedicated placeholder score (its top known-class logit minus its top dummy-unit logit). For every score, we calibrate a rejection threshold at the 5th percentile of in-distribution validation scores, and report AUROC, a threshold-free measure of how well the score separates knowns from unknowns, separately for near-unknowns, far-unknowns, and the two pooled together.

Results

Method	Score	Closed-Set Acc	AUROC(near)	AUROC(far)	AUROC(all)
Vanilla	MSP	94.58%	0.806	0.890	0.848
Vanilla	MLS	94.58%	0.787	0.889	0.838
Vanilla	Energy	94.58%	0.787	0.889	0.838
Vanilla	Mahalanobis	94.58%	0.806	0.912	0.859
GCSC	MLS	86.94%	0.730	0.878	0.804
PROSER	MLS	94.24%	0.782	0.874	0.828
PROSER	Placeholder	94.24%	0.807	0.927	0.867

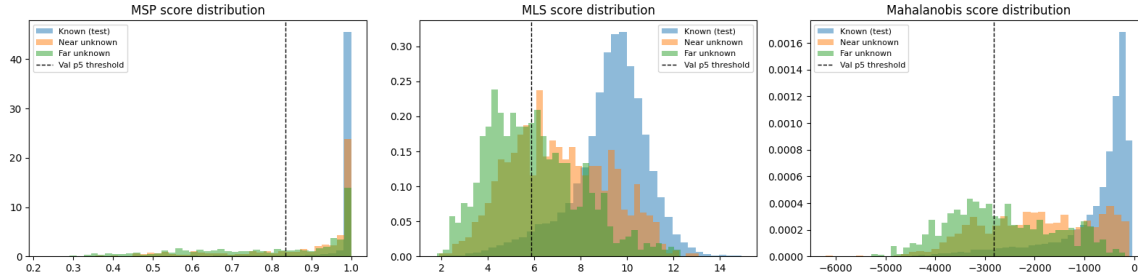


Figure: Vanilla-model novelty-score distributions (known test / near-unknown / far-unknown) for MSP, MLS, and Mahalanobis. MSP piles up near 1.0 for known images and for a large share of the unknowns too; Mahalanobis pushes far-unknown furthest left while near-unknown stays entangled with the known-test mass, the visual counterpart of the near/far AUROC gap in the table above.

GCSC's heavier augmentation doesn't pay off: it costs 7.6 points of closed-set accuracy compared to Vanilla (86.94% vs. 94.58%), so it never becomes the stronger closed-set classifier the idea relies on, and it also produces the *worst* near and pooled AUROC of any method, including the lowest near-AUROC in the whole table (0.730); only on far-OOD does PROSER's MLS score do slightly worse (0.874 vs. 0.878).

PROSER's dedicated placeholder score is the best detector overall (0.867 AUROC across all unknowns), better than its own MLS score on the exact same backbone (0.828), and better than every score computed on the plain Vanilla model, all at a tiny 0.34-point cost to closed-set accuracy.

Every single method/score combination we tried is noticeably worse at catching near-unknowns than far-unknowns (a gap of 0.08–0.15 AUROC). This suggests that how *semantically close* an unknown class is to the known classes is what drives how hard it is to detect. This gap showing up consistently across seven different method/score combinations, regardless of how the classifier was trained, suggests it's a property of the near/far split. At the validation-calibrated threshold (which keeps about 95% of known test images), even the best detectors only reject about a quarter of near-unknowns (PROSER's placeholder 26.8%, Mahalanobis 24.1%), and plain MLS actually rejects more of them (32.6%), so the AUROC ranking doesn't carry over to this operating point for near-OOD. On far-OOD, PROSER's placeholder score still rejects the most (60.3%).

Discussion

The training objective not matching the final result.

In both Tasks 2 and 3, we cannot optimize target accuracy directly because the target-domain labels are not available during training, so we instead rely on other objectives to improve generalization. DAN/DANN/CDAN push the feature distributions toward looking domain-invariant; SAM pushes the loss landscape toward being flat. CDAN pushed its domain classifier down to chance and DANN's adversarial training blew up, yet both failed badly on Sketch (30.54%/14.34% and 35.71%/37.05% accuracy/macro-F1, against a 72.46% accuracy baseline with no adaptation at all), and SAM found the flattest minimum of the three Task 3 methods ($\Delta_{\text{sharp}}=0.2059$, vs. ERM's 0.4146) while producing the *worst* Sketch accuracy of the three. Domain-invariance and flatness are both reasonable things to optimize for in principle, but neither one guarantees good out-of-domain accuracy, and both tasks show the proxy can actively make things worse.

Knowing what we do and not know.

Tasks 1 through 3 all assume every test input belongs to one of the known classes, and just ask *which one*. Task 4 drops that assumption and asks a logically prior question: does this input belong to the known set *at all*? The ordinary closed-set classifier we tested in Task 4 doesn't give a good answer to that question for free. Plain softmax confidence (MSP) did reasonably (0.848 pooled AUROC) but still trailed something more deliberate, Mahalanobis distance over class-conditional feature statistics (0.859), and the best overall AUROC (0.867) needed extra training through PROSER's placeholder score. The near/far split adds that every method's AUROC was 8–15 points worse on near-OOD classes (ones semantically close to CIFAR-10 classes) than on far-OOD classes, so "is this in-distribution?" is only an easy question when the novel class is semantically far from the known ones.

Two unrelated methods, the same wrong answer.

As noted in the Task 3 Results, both DANN (Task 2) and SAM (Task 3) end up over-predicting the same class, "dog", on Sketch, even though they share no mechanism (one is an adversarial classifier, the other an optimizer-level flatness penalty). We don't read much into this, though. DANN's run diverged, so its collapse says more about unstable training than about Sketch, and SAM still gets 39–86% of every class right rather than collapsing. Whether "dog" really is a default answer for weak Sketch classifiers would need follow-up we didn't run, such as checking class frequencies and how detailed each class's sketches are.

A possible reason Sketch is hard for everyone.

Task 1 found that ResNet-50 has a shape bias of only 53.2% (compared to ViT's 82.6% and CLIP's roughly 80%), meaning that when shape and texture disagree, it sides with texture almost as often as with shape. Every method in Tasks 2 and 3 uses a ResNet-18 backbone, and PACS's Sketch domain is precisely the domain that strips out texture almost entirely, leaving only outlines and shape. Since we didn't directly measure shape bias on the actual Task 2/3 backbones, we can only hypothesize: Sketch being so hard across *every* method in both tasks is consistent with a texture-reliant backbone being fundamentally mismatched with a domain whose only real signal is shape, no matter what adaptation or generalization trick is layered on top. If this is right, it suggests that on PACS-Sketch specifically, the backbone's inductive bias may be capping how much any amount of adversarial alignment or flatness regularization can actually help.

Conclusion

Across four tasks that each broke the standard train/test assumption in a different way, a few lessons kept showing up. Task 1 showed that the CNN leans on texture far more than the two transformers, and that shape bias doesn't predict robustness: ViT and CLIP are similarly shape-biased, yet one is the most robust to patch shuffling and the other the least. Task 2 showed that pushing a domain classifier toward confusion can sink target-domain accuracy instead of increasing it, with the gap between DANN's raw accuracy and macro-F1 exposing a class-collapse that accuracy alone would have hidden. Task 3 showed the same kind of decoupling for a completely different regularizer: the flattest loss landscape did not produce the best accuracy on a genuinely unseen domain. Task 4 showed that ordinary closed-set classifiers don't get open-set rejection for free, you need deliberately chosen machinery (PROSER's placeholder score did best, and Mahalanobis distance was the best of the post-hoc scores), and that spotting an unknown class is only easy when that class is also semantically distant from anything the model has seen. Overall, Tasks 2 and 3 show that doing better on a secondary objective or diagnostic does not always translate into better final performance.

References

1. Geirhos, R., Rubisch, P., Michaelis, C., Bethge, M., Wichmann, F. A., & Brendel, W. (2019). ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. *ICLR*.
2. Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., & Lempitsky, V. (2016). Domain-adversarial training of neural networks. *JMLR*. (DANN)
3. Long, M., Cao, Z., Wang, J., & Jordan, M. I. (2018). Conditional adversarial domain adaptation. *NeurIPS*. (CDAN)
4. Long, M., Cao, Y., Wang, J., & Jordan, M. I. (2015). Learning transferable features with deep adaptation networks. *ICML*. (DAN)
5. Foret, P., Kleiner, A., Mobahi, H., & Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. *ICLR*. (SAM)
6. Zhou, K., Liu, Z., Qiao, Y., Xiang, T., & Loy, C. C. (2022). Domain generalization: A survey. *IEEE TPAMI*.
7. Zhou, D.-W., Ye, H.-J., & Zhan, D.-C. (2021). Learning placeholders for open-set recognition. *CVPR*. (PROSER)
8. Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *ICLR*. (MSP)
9. Liu, W., Wang, X., Owens, J., & Li, Y. (2020). Energy-based out-of-distribution detection. *NeurIPS*.
10. Lee, K., Lee, K., Lee, H., & Shin, J. (2018). A simple unified framework for detecting out-of-distribution samples and adversarial attacks. *NeurIPS*. (Mahalanobis)
11. Gatys, L. A., Ecker, A. S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. *CVPR*.
12. Huang, X., & Belongie, S. (2017). Arbitrary style transfer in real-time with adaptive instance normalization. *ICCV*. (AdaIN)
13. Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision. *ICML*. (CLIP)
14. Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*. (ViT)
15. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*. (ResNet)
16. Zhao, H., des Combes, R. T., Zhang, K., & Gordon, G. (2019). On learning invariant representations for domain adaptation. *ICML*.
17. Vaze, S., Han, K., Vedaldi, A., & Zisserman, A. (2022). Open-set recognition: A good closed-set classifier is all you need. *ICLR*. (GCSC)